

# 算法设计与分析讲义

---

Lecture notes on Algorithm Design and Analysis (LADA)

黄宇

南京大学计算机科学与技术学院

<https://github.com/alg-nju/lada>

2026 年 8 月 18 日

# 使用说明

- 讲义的一章大致对应于一次授课。每一章的开头会有一个内容概要，列出此次授课所有的知识要点。
- 讲义的内容在持续写作中。目前优先写进讲义的内容，是当前授课所用的课本和胶片中，讲得不足需要补充和调整的部分。也就是说，讲义尚未写出来的内容，在课本和胶片中可以找到相应的详细描述。

# 前言

前言。

[1]。

# 目录

|                                       |           |
|---------------------------------------|-----------|
| <b>第一部分 准备知识</b>                      | <b>4</b>  |
| <b>第一章 计算模型</b>                       | <b>6</b>  |
| 1.1 抽象的计算机：计算模型的引入 . . . . .          | 6         |
| 1.1.1 算法的直观概念 . . . . .               | 6         |
| 1.1.2 Pointer模型：RAM模型的对象化改造 . . . . . | 6         |
| 1.2 抽象的算法设计 . . . . .                 | 7         |
| 1.3 抽象的算法分析 . . . . .                 | 7         |
| <b>第二章 函数的渐近增长率</b>                   | <b>9</b>  |
| <b>第三章 分治递归求解</b>                     | <b>10</b> |
| 3.1 Master定理 . . . . .                | 10        |
| <b>第二部分 从图遍历到图优化</b>                  | <b>13</b> |
| <b>第四章 图遍历</b>                        | <b>15</b> |
| 4.1 图遍历的抽象框架 . . . . .                | 15        |

# 第一部分

## 准备知识

准备知识部分。

# 第一章 计算模型

## 1.1 抽象的计算机：计算模型的引入

### 1.1.1 算法的直观概念

我们首先不严格地讨论算法的直观概念。

算法是“the step-by-step procedures that computers use to solve problems”<sup>1</sup>。

我们可以借用“类比”（analog）来解释算法相关的概念<sup>2</sup>。假设我们了解一台计算机及其上运行的程序，那么我们的“算法”，就是“计算机程序在数学世界中的类比”（mathematical analog of a computer program）。我们描述算法所用的“伪码”，就是“计算机编程语言在数学世界中类比”（mathematical analog of a programming language）。这里所说的伪码是指面向算法描述的，结构化的自然语言，可以是中文、英文等。我们的“计算模型”，就是“现实世界中的计算机在数学世界中的类比”（mathematical analog of a computer）。

表 1.1: 概念的对比

| 数学世界的概念 | 计算机世界的概念 |
|---------|----------|
| 算法      | 程序       |
| 伪码语言    | 编程语言     |
| 计算模型    | 计算机      |

### 1.1.2 Pointer模型：RAM模型的对象化改造

在算法设计与分析的过程中，我们经常需要用到动态分配的数据对象，例如各种链表、以邻接链表表示的图等。基础的RAM模型并不直接支持“寻址”的概念，表达动态数据结构的时候，会非常地不方便。为了便于使用各种常用的动态数据结构，我们引入Pointer模型的概念<sup>3</sup>。

在Pointer模型中，我们可以动态分配、修改、释放数据对象（object）。每个数据对象，包含若干个（但是不超过一个常数的上界）字段（field），每个字段可以是一份存储单元中的数据，

<sup>1</sup>这一说法源自于Quanta Magazine的文章《Computer scientists establish the best way to traverse a graph》，作者Ben Brubaker，链接：<https://www.quantamagazine.org/computer-scientists-establish-the-best-way-to-traverse-a-graph-20241025/>。

<sup>2</sup>这里参考了Erik Demaine教授在课堂上的讲授。课程信息：MIT open courseware, Introduction to algorithms (6.006, fall 2011)。链接：<https://courses.csail.mit.edu/6.006/fall11/notes.shtml>。最后访问日期：2026.8.5。

<sup>3</sup>同样参考了Erik Demaine教授在MIT 6.006算法课程中的相关讲解，详见§1.1.1的脚注。

也可以是一个指针（pointer），指针指向另一个数据对象——这也是Pointer模型名字的由来。我们还特别约定指针可以取特殊值NULL，表示其不指向任何数据对象。

我们可以在基本的RAM模型中，实现上述指针模型。简单而言，我们可以用存储单元的序号（index）来表示对象的位置，进而实现指针的概念。在实际使用中，我们不详细讨论实现细节，只是假设我们的RAM模型具有动态分配数据对象，寻址一个数据对象并进行各种操作的能力。

## 1.2 抽象的算法设计

## 1.3 抽象的算法分析

### 章节注释

#### 算法（algorithm）一词的由来

花拉子米（780年代-850年代<sup>4</sup>），波斯数学家。他的《代数学》是在约830年写成的一部数学著作，这是第一本解决一次方程及一元二次方程的系统著作，他因而被称为代数的创造者。代数的英文单词“algebra”就出自于他的著作中对于一元二次方程的一种解法。花拉子米在825年写成的《印度数字算术》（On the Calculation with Hindu Numerals）一书对于印度-阿拉伯数字系统在中东及欧洲的传播尤其重要。《印度数字算术》被翻译成拉丁语“Algoritmi de numero Indorum”，而“算法”（algorithm）一词则来自于花拉子米名字的拉丁文音译。

#### 是“模型源自于实践”，还是“模型指导了实践”

在我们的授课过程中，对于算法知识的学习和编程实践之间的关系，我们是有预设的要求的。我们的授课，假设大家有基本的编程知识，并且有过一定量的基础编程实践。在此基础上，希望我们的算法设计和分析课程能够帮助大家更好地认识到程序的正确运作、高效运作背后的原理。基于这一“设定”，这次课所讨论的作为抽象算法设计与分析之基础的计算模型，更像是从具体的实践中归纳出来的。

深入思考上面的设定，大家有可能会有这样的困惑。讨论算法的源头，一般都会提到欧几里得<sup>5</sup>算法，它大概形成于中国的战国时期。中国的三国时期的数学家刘徽所提出的割圆法，也是一种优美的算法。为什么在一个电都没有的年代，却可以有算法。另一个更著名的算法问题是希尔伯特第十问题<sup>6</sup>，它要求寻找判定丢番图方程是否有解的算法。但究竟什么是算法呢？在希尔伯特提出问题之时却并不存在一个明确定义。

最终图灵机的出现，对什么是计算，什么是算法，给出了精确的定义。图灵机首先是一个纯粹的数学抽象，其诞生动机来自数理逻辑内部的问题，而非当时物理计算技术的直接反映。如果非要说有什么“得益于”的地方，那也许是当时的打字机、磁带等日常机械给了图灵关于“读写头+移动纸带”这一直观隐喻。倒是物理的计算机的制造，得益于图灵机相关理论概念的发展。图

<sup>4</sup>大约是中国历史上安史之乱之后，唐朝中晚期时代。

<sup>5</sup>Euclid，公元前325年——公元前265年

<sup>6</sup>关于希尔伯特第十问题的材料有多种，这里主要参考了维基百科和卢昌海老师的《Hilbert第十问题漫谈》



灵机先定义了通用计算的数学边界，后来的冯·诺依曼架构、存储程序计算机，在概念上受到了图灵抽象的间接影响。图灵本人二战后参与建造的 ACE (Automatic Computing Engine) 明确借鉴了他在1936 年提出的图灵机相关的理论。

所以回到我们的这次课程。从计算机科学技术的历史来看，我们更应该理解为抽象的模型指导了具体的实践。但是在我们今天学习这门本科生基础课程的时候，我们已经站在了巨人的肩膀上。此时，我们的授课方法是，以充分的实践为诱因和助力，更好得学习和理解算法的知识。

## 第二章 函数的渐近增长率

### 章节注释

#### 微积分严格化的历史

$\epsilon - \delta$ 语言的背后，是微积分严格化的艰苦历程。

牛顿和莱布尼茨虽然把微积分系统化，但是它还是不够严谨。可是当微积分被成功地用来解决许多问题，却使得十八世纪的数学家偏向其应用，而甚少致力于其严谨性。一些数学家，包括科林·麦克劳林，试图证明使用无穷小是可靠的做法，但直到150多年之后才得以成功。

奥古斯丁·路易·柯西和卡尔·魏尔斯特拉斯的工作，实现了对无穷小的记号的回避。微分和积分的基础终于被打下了。在柯西的著作中，可以看到一系列的基础进路尝试，包括通过无穷小来对连续进行定义，和在微分定义中一个不太精确的函数极限原型。而在魏尔斯特拉斯的著述中，对极限概念作了形式化，回避了无穷小的使用。继魏尔斯特拉斯之后，微积分就常以极限作为基础，而非无穷小了。

## 第三章 分治递归求解

### 3.1 Master定理

**定理 3.1** (Master定理). 令 $a \geq 1$ 和 $b > 1$ 为常数,  $f(n)$ 为一定义于非负整数上的函数。 $T(n)$ 为定义于非负整数上的递归函数:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \quad (3.1)$$

递归式中的 $\frac{n}{b}$ 指的是 $\lfloor \frac{n}{b} \rfloor$ 或者 $\lceil \frac{n}{b} \rceil$ 。 $T(n)$ 的渐近增长率为:

1. 如果存在某个常数 $\epsilon > 0$ , 使得 $f(n) = O(n^{\log_b a - \epsilon})$ , 则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 如果 $f(n) = \Theta(n^{\log_b a})$ , 则 $T(n) = \Theta(n^{\log_b a} \log n)$ 。
3. 如果存在常数 $\epsilon > 0$ , 使得 $f(n) = \Omega(n^{\log_b a + \epsilon})$ , 且存在某个常数 $c < 1$ , 使得对所有充分大的 $n$ ,  $af(\frac{n}{b}) \leq cf(n)$ , 则 $T(n) = \Theta(f(n))$ 。

证明. 根据图3.1, 我们有:

$$T(n) = \underbrace{\Theta(n^{\log_b a})}_{\text{叶节点层代价}} + \underbrace{g(n)}_{\text{所有非叶结点层代价}} \quad (3.2)$$

$$g(n) = \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j f\left(\frac{n}{b^j}\right) \quad (3.3)$$

对于 $g(n)$ 的求和形式, 注意当 $j$ 取0时, 我们得到 $f(n)$ 。此外, 求和的项数必须是整数。递归树的层数记为 $\lfloor \log_b n \rfloor + 1$ , 则非叶子节点有 $\lfloor \log_b n \rfloor$ 层, 即 $g(n)$ 的求和形式有 $\lfloor \log_b n \rfloor$ 项。

**第一种情况.** 已知 $f(n) = O(n^{\log_b a - \epsilon})$ , 所以 $f(\frac{n}{b^j}) = O((\frac{n}{b^j})^{\log_b a - \epsilon})$ 。据此, 我们有 $g(n)$ 满足:

$$\begin{aligned}
 g(n) &= \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon} \\
 &= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor \log_b n \rfloor} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\
 &= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor \log_b n \rfloor} (b^\epsilon)^j \\
 &= n^{\log_b a - \epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right) \\
 &= n^{\log_b a - \epsilon} \left(\frac{n^{\epsilon - 1}}{b^\epsilon - 1}\right)
 \end{aligned}$$

上面的推导过程中, 为了表示的方便, 我们省去了 $g(n)$ 表达式中的渐进增长率符号 $O(\cdot)$ 。因为 $b$ 和 $\epsilon$ 为常数, 所以我们有 $g(n) = O(n^{\log_b a - \epsilon} n^\epsilon) = O(n^{\log_b a})$ 。所以根据公式3.2, 我们证明了 $T(n) = \Theta(n^{\log_b a})$ 。

**第二种情况.** 已知 $f(n) = \Theta(n^{\log_b a})$ , 则 $f(\frac{n}{b^j}) = \Theta((\frac{n}{b^j})^{\log_b a})$ 。据此, 我们有 $g(n)$ 满足:

$$\begin{aligned}
 g(n) &= \sum_{j=0}^{\lfloor \log_b n \rfloor} a^j \left(\frac{n}{b^j}\right)^{\log_b a} \\
 &= n^{\log_b a} \sum_{j=0}^{\lfloor \log_b n \rfloor} \left(\frac{a}{b^{\log_b a}}\right)^j \\
 &= n^{\log_b a} \sum_{j=0}^{\lfloor \log_b n \rfloor} 1 \\
 &= n^{\log_b a} \log_b n
 \end{aligned}$$

上面的推导过程中, 为了表示的方便, 我们同样省去了 $g(n)$ 表达式中的渐进增长率符号 $\Theta(\cdot)$ 。根据公式3.2, 我们证明了 $T(n) = \Theta(n^{\log_b a} \log n)$ 。

**第三种情况.** 根据 $T(n)$ 的原始表达式 (公式3.1), 我们知道 $T(n) = \Omega(f(n))$ 。所以, 为了证明 $T(n) = \Theta(f(n))$ , 我们的任务只剩下证明 $T(n) = O(f(n))$ 。根据第三种情况的第一个条件: 存在常数 $\epsilon > 0$ , 使得 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 。根据 $T(n)$ 的递归树展开表达式 (公式3.2), 我们有 $T(n)$ 的第一项 $n^{\log_b a} = O(f(n))$ 。此时我们的任务只剩下证明 $T(n)$ 的第二项 $g(n) = O(f(n))$ 。而这一结论, 完全是第三种情况的第二个条件, 给“硬性规定出来”的。

对于第三种情况, 已知 $af(\frac{n}{b}) \leq cf(n)$ , 所以 $a^2 f(\frac{n}{b^2}) = a \cdot af(\frac{n}{b}) \leq a \cdot cf(\frac{n}{b}) \leq c^2 f(n)$ 。上



## 第二部分

### 从图遍历到图优化

图遍历。

图优化：贪心、动态规划。

## 第四章 图遍历

### 4.1 图遍历的抽象框架

WHATEVERFIRSTSEARCH (WFS), 算法1。

---

**Algorithm 1** WHATEVERFIRSTSEARCH( $s$ )

---

```
1: put  $s$  into the bag  $B$ 
2: while  $B$  is not empty do
3:   take  $v$  from  $B$ 
4:   if  $v$  is unmarked then
5:     mark  $v$ 
6:     for all edge  $vw$  do
7:       put  $w$  into  $B$ 
```

---

WFS的外层调用算法 (wrapper), 算法2。

---

**Algorithm 2** WFSWRAPPER( $G$ )

---

```
1: for all vertices  $v$  do unmark  $v$ 
2: for all vertices  $v$  do
3:   if  $v$  is unmarked then WFS( $v$ )
```

---



## 参考文献

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Ctein. *Introduction to Algorithms (Fourth edition)*. the MIT Press, 2022.